

RAPPORT — LOUIS DROUHIN, OWEN BOUDON, SÉBASTIEN RUET, ZAKARIYA SAOULA

Breezy

Réseau social distribué en architecture microservices

Rapport de projet — Développement d'applications distribuées

Table des matières

1	Introduction	2
1.1	Contexte du module et du projet	2
1.2	Objectifs et attentes du projet	2
1.3	Présentation de l'équipe	2
1.4	Périmètre retenu et hors-scope justifié	2
2	Analyse des besoins et fonctionnalités	4
2.1	User stories et hiérarchisation	4
2.2	Matrice des permissions	4
2.3	Méthodologie et gestion de projet	4
2.4	Détail des étapes et ressources mobilisées	5
3	Architecture du système	6
3.1	Vue d'ensemble et justification du choix microservices	6
3.2	Schéma d'architecture	6
3.3	Stack technique par service	7
3.4	Modèle de données et cohérence inter-services	7
3.5	Sécurité et gateway Nginx	7
3.6	Conteneurisation	8
4	Réalisation	9
4.1	auth-svc : Authentification et JWT (Louis DROUHIN)	9
4.2	user-svc : Profils utilisateurs (Sébastien RUET)	9
4.3	post-svc : Publications et commentaires (Zakariya SAOULA)	9
4.4	notif-svc : Notifications (Owen BOUDON)	9
4.5	feed-svc : Fil d'actualité (Louis DROUHIN)	10
4.6	Gateway Nginx (Sébastien RUET)	10
4.7	Front-end (Louis DROUHIN, Sébastien RUET)	10
5	Difficultés rencontrées et solutions	11
5.1	Choix techniques discutés en équipe	11
5.2	Points de friction inter-services	11
6	Conclusion	12
6.1	Bilan par rapport aux objectifs	12
6.2	Axes d'amélioration et évolutions potentielles	12

1. Introduction

1.1. Contexte du module et du projet

Ce rapport présente le travail réalisé dans le cadre du projet Fil Rouge du module *Développement d'applications distribuées*. Ce projet consiste à développer **Breezy**, un réseau social léger inspiré de Twitter/X.

Celui-ci se doit d'être robuste et scalable, en s'appuyant sur une architecture en microservices plutôt que sur un monolithe applicatif.

1.2. Objectifs et attentes du projet

Le cahier des charges fourni en début de séquence définit Breezy comme un réseau social inspiré de Twitter, dont le périmètre fonctionnel est découpé en fonctionnalités primaires (obligatoires) et secondaires (optionnelles). Les fonctionnalités primaires couvrent le socle minimal d'un réseau social viable :

- création de compte et authentification sécurisée ;
- publication de messages courts (280 caractères) ;
- affichage des messages sur le profil de l'utilisateur ;
- fil d'actualité chronologique des comptes suivis ;
- interactions sociales de base : like, commentaire, réponse à un commentaire, follow/unfollow ;
- profil utilisateur avec informations de base.

1.3. Présentation de l'équipe

Membre	Responsabilité principale
Louis DROUHIN	auth-svc (authentification, JWT), feed-svc, et front-end
Sébastien RUET	user-svc (profils, follow), gateway Nginx et front-end
Owen BOUDON	notif-svc (notifications)
Zakariya SAOULA	post-svc (publications, commentaires)

Table 1 : Répartition des responsabilités au sein de l'équipe

Cette répartition suit le découpage naturel de l'architecture en microservices : à partir du moment où les contrats d'API entre services ont été fixés, chaque membre a pu avancer en parallèle sur son service sans dépendance bloquante sur le travail des autres, à l'exception du front-end qui consomme l'ensemble des API.

1.4. Périmètre retenu et hors-scope justifié

Le cahier des charges propose un large éventail de fonctionnalités secondaires (messagerie privée, médias riches, signalement de contenu, interface multi-

langues, thèmes personnalisés, suspension de comptes...). Conscients de la charge de travail que représenterait leur implémentation complète au regard du temps imparti, nous avons fait le choix de retirer volontairement plusieurs fonctionnalités du périmètre, plutôt que de les implémenter partiellement ou de manière peu aboutie.

Fonctionnalités non développées, et raisons :

- **Vidéos sur les posts** : complexité de stockage et de traitement de fichiers non proportionnée au bénéfice pour cette itération, cependant les images et GIF sont pris en charge ;
- **Signalement de contenu et modération (suppression de posts, bannissements)** : nécessite un back-office supplémentaire et donc du temps supplémentaire, qui pourra cependant être développé dans un second temps ;
- **Thème sombre** : complexité de la conversion d'une palette de couleur en mode sombre, qui n'apporte pas de valeur fonctionnelle directe pour cette itération ;
- **Messagerie privée** : fonctionnalité complète à part entière (modèle de données, chiffrement éventuel) repoussée en évolution future.

Ce choix de périmètre a été tranché collectivement et documenté dès le début du développement, afin qu'aucun membre de l'équipe ne réintroduise une de ces fonctionnalités par anticipation sans concertation.

2. Analyse des besoins et fonctionnalités

2.1. User stories et hiérarchisation

Les fonctionnalités du cahier des charges sont exprimées sous forme de user stories (« En tant que..., je veux..., afin de... »), ce qui a permis de prioriser le développement par criticité plutôt que par ordre d'apparition dans le document.

Code	Fonctionnalité	Priorité
Fx1	Création de compte avec validation	Primaire
Fx2	Authentification sécurisée (JWT)	Primaire
Fx3	Publication de messages courts	Primaire
Fx4	Affichage des messages sur le profil	Primaire
Fx5	Fil chronologique des comptes suivis	Primaire
Fx6	Like d'un post	Primaire
Fx7	Commentaire sur un post	Primaire
Fx8	Réponse à un commentaire	Primaire
Fx9	Follow / unfollow	Primaire
Fx10	Profil utilisateur (bio, avatar)	Primaire
Fx11	Liste des messages publiés sur le profil	Primaire
Fx13	Recherche par tags	Secondaire
Fx14–Fx16	Notifications (mention, like, follower)	Secondaire
Fx18	Ajout d'images aux messages	Secondaire
Fx22	Interface multi-langues	Secondaire

Table 2 : Hiérarchisation retenue des fonctionnalités

2.2. Matrice des permissions

La gestion des droits repose sur un rôle porté par chaque compte (`user` ou `admin`), vérifié à la gateway. Le tableau suivant synthétise les accès retenus pour les fonctionnalités implémentées.

Fonctionnalité	Visiteur	Utilisateur authentifié
Création de compte	Oui	—
Authentification	Oui	Oui
Publication de messages	Non	Oui
Consultation des profils/posts	Non	Oui
Like, commentaire, réponse	Non	Oui
Follow / unfollow	Non	Oui

Table 3 : Matrice des permissions appliquée au périmètre retenu

2.3. Méthodologie et gestion de projet

L'équipe s'est appuyée sur des standards tout au long du développement :

- un modèle de branches **Gitflow** (main, develop, feature/<service>-<description>, fix/<description>), chaque fonctionnalité étant développée à partir de develop et jamais directement sur main;
- des messages de commit normalisés selon la convention **Conventional Commits** (feat, fix, docs, refactor, test, chore), avec le service concerné en *scope* (ex. feat(auth-svc): ajoute la route de login);
- un **board Kanban** à trois colonnes principales (À faire / En cours / Terminé), complété d'une colonne Blocage pour signaler les dépendances entre services;
- un **dépôt privé** partagé entre les quatre membres.

2.4. Détail des étapes et ressources mobilisées

Le projet a suivi la progression du module : modélisation de l'architecture (boucle API), développement du back-end REST par service, intégration de la sécurité JWT et de la conteneurisation Docker, puis développement du front-end React/Next.js consommant les API exposées par la gateway.

Catégorie	Ressources mobilisées
Langage / runtime	JavaScript / TypeScript, Node.js
Back-end	Express.js, Sequelize, Mongoose, jsonwebtoken
Bases de données	PostgreSQL (auth-svc, user-svc), MongoDB (post-svc, notif-svc)
Front-end	React, Next.js
Infrastructure	Docker, Docker Compose, Nginx (reverse proxy)
Outillage	pnpm (monorepo), Git, GitHub (dépôt, Issues, Projects)

Table 4 : Ressources techniques mobilisées pour le projet

3. Architecture du système

3.1. Vue d'ensemble et justification du choix microservices

Le sujet imposait de réfléchir au choix d'une architecture logicielle adaptée, entre monolithique et microservices. Nous avons retenu une architecture en microservices, chaque service disposant de sa propre base de données, pour les raisons suivantes :

- **isolation des responsabilités** : authentification, gestion de profils, publications et notifications sont des domaines fonctionnels distincts, avec des contraintes de cohérence et de volumétrie différentes ;
- **parallélisation du développement** : le découpage par service a permis à chaque membre de l'équipe de travailler de façon autonome dès que les contrats d'API ont été stabilisés ;
- **portabilité** : chaque service est conteneurisé indépendamment, ce qui facilite un futur déploiement sur un orchestrateur (cf. axes d'amélioration).

3.2. Schéma d'architecture

Architecture retenue : un point d'entrée HTTP unique (Nginx) route les requêtes vers cinq services métier, isolés du frontend sur leur propre réseau docker `backend` et disposant (sauf `feed-svc`) de sa propre base de données.

Service	Rôle	Port par défaut
<code>auth-svc</code>	Authentification, JWT	3001
<code>user-svc</code>	Profils, relations follow	3002
<code>post-svc</code>	Publications, commentaires	3003
<code>notif-svc</code>	Notifications	3004
<code>feed-svc</code>	Fil d'actualité (sans base)	3005
Frontend Next.js	Interface utilisateur	3000
Nginx (gateway)	Point d'entrée, reverse proxy	80

Table 5 : Services et ports de l'architecture Breezy

Le client ne dialogue jamais directement avec un service métier : toutes les requêtes transitent par Nginx, qui les redirige selon le préfixe de route (`/api/auth`, `/api/users`, `/api/posts`, `/api/notifications`, `/api/feed`). Aucun service backend n'est exposé directement à l'extérieur du réseau Docker.

3.3. Stack technique par service

Service	Moteur	ORM/ODM	Raison du choix
auth-svc	PostgreSQL	Sequelize	Données critiques, contraintes fortes, transactions ACID
user-svc	PostgreSQL	Sequelize	Le graphe de follow est un many-to-many relationnel classique
post-svc	MongoDB	Mongoose	Volume d'écriture élevé, structure semi-flexible
notif-svc	MongoDB	Mongoose	Fort volume d'écriture, payload variable selon le type de notification
feed-svc	—	—	Calculé à la volée, pas de persistance propre

Table 6 : Choix technologiques par service et justification

3.4. Modèle de données et cohérence inter-services

Le `username` constitue la clé de cohérence partagée entre tous les services :

- Il est immuable : aucune route de modification de `username` n'existe, dans aucun service ;
- Il est la clé primaire des entités utilisateur (`Account` dans `auth-svc`, `Profile` dans `user-svc`) ;
- Les services métier (`post-svc`, `notif-svc`) ne font aucune clé étrangère technique vers `auth-svc/user-svc` (impossible de toute façon entre bases de données différentes) mais stockent simplement le `username` en tant que champ simple (`authorUsername`, `recipientUsername`) ;
- La cohérence est donc applicative et non technique : c'est `auth-svc` qui garantit l'unicité du `username` à la création de compte, les autres services lui font confiance.

Un choix de modélisation notable concerne les commentaires dans `post-svc` : un commentaire est un `Post` comme un autre, simplement doté d'un champ `parentId` pointant vers le post auquel il répond. Ce choix évite de dupliquer un modèle `Comment` séparé et permet nativement des fils de discussion à profondeur illimitée.

3.5. Sécurité et gateway Nginx

Conformément aux standards vus dans le prosit JWT/Docker, l'authentification repose sur un mécanisme JWT délégué :

1. Nginx intercepte chaque requête entrante et déclenche une directive `auth_request` ;
2. cette directive délègue la validation effective du jeton à `auth-svc`, via une route dédiée (`/api/auth/validate`) ;

3. si le jeton est valide, Nginx transmet aux services métier deux en-têtes de confiance : `X-User-Username` et `X-User-Role`;
4. les services métier (`post-svc`, `notif-svc`, `user-svc`, `feed-svc`) ne réimplémentent pas de logique de vérification JWT : ils font confiance aux en-têtes transmis par la gateway après validation.

Gateway = Nginx pur. Aucun service Node.js personnalisé ne fait office d'API Gateway dans ce projet : Nginx, configuré en reverse proxy, assure à lui seul le routage et la délégation de l'authentification. Ce choix limite la surface de code à maintenir et s'appuie sur un composant éprouvé plutôt que de réinventer une brique de sécurité.

Les autres règles de sécurité transverses appliquées à l'ensemble des services Node.js sont : gestion d'erreurs centralisée (middleware d'erreur Express, jamais de stack trace brute renvoyée au client), CORS configuré explicitement, variables sensibles exclusivement via `dotenv`, et respect du niveau 2 du modèle de maturité de Richardson (verbes HTTP sémantiquement corrects, codes de statut HTTP).

3.6. Conteneurisation

L'ensemble des services est orchestré via Docker Compose, avec un fichier dédié au développement (hot-reload par bind-mount du code source) et un fichier dédié à la production (`docker-compose.prod.yml`, images distinctes par microservice, construites et hébergées sur le Docker Registry de Github via Github Actions). Deux réseaux Docker isolent le frontend du backend, Nginx faisant le pont entre les deux. Des *profiles* Docker Compose permettent de ne démarrer que les services utiles pour du développement ciblé plutôt que l'intégralité de la stack.

4. Réalisation

Cette section détaille, service par service, les fonctionnalités implémentées et les choix de mise en œuvre. Chaque sous-section a été rédigée par le ou les membres ayant porté le développement du service correspondant.

4.1. `auth-svc` : Authentification et JWT (Louis DROUHIN)

`auth-svc` couvre Fx1 (création de compte) et Fx2 (authentification sécurisée). Il ne stocke que les informations strictement nécessaires à l'authentification : `username`, `email`, `passwordHash`, `role`, `active`. Toute donnée de profil (bio, avatar) relève de `user-svc`.

- inscription avec hachage du mot de passe (jamais stocké en clair) ;
- connexion délivrant un jeton JWT signé, vérifié ensuite côté Nginx via `auth_request` ;
- déconnexion par suppression de la ligne de refresh token en base, sans champ de révocation dédié, choix retenu pour limiter la complexité du modèle de données au regard du périmètre du projet.

4.2. `user-svc` : Profils utilisateurs (Sébastien RUET)

`user-svc` couvre Fx9 (follow/unfollow) et Fx10 (profil utilisateur). Le graphe de relations de follow est modélisé de façon relationnelle classique (table d'association many-to-many), ce qui justifie le choix de PostgreSQL plutôt que MongoDB pour ce service.

4.3. `post-svc` : Publications et commentaires (Zakariya SAOULA)

`post-svc` couvre Fx3 (publication de messages courts), Fx4 (affichage sur le profil), Fx6 (like), Fx7 et Fx8 (commentaires et réponses). Le choix de MongoDB pour ce service est justifié par un volume d'écriture potentiellement élevé et une structure de données semi-flexible (un post peut porter un nombre variable de likes, de tags, de réponses).

Le point de mise en œuvre central est le modèle unique `Post` avec champ `parentId` optionnel : un post racine a un `parentId` nul, un commentaire référence le post auquel il répond, et une réponse à un commentaire référence ce commentaire, sans limite de profondeur. Le compteur de réponses (`replyCount`) et de like (`likeCount`) est recalculé à chaque ajout/suppression pour éviter un comptage à la volée coûteux sur des fils profonds.

4.4. `notif-svc` : Notifications (Owen BOUDON)

`notif-svc` couvre Fx14 à Fx16 (notifications de mention, de like, de nouveau follower). Le choix de MongoDB se justifie par un fort volume d'écriture et un *payload* variable selon le type de notification (une notification de mention ne porte pas les mêmes champs qu'une notification de like).

Le service expose une route de consultation des notifications d'un utilisateur (authentifié via les en-têtes transmis par la gateway) ainsi qu'une route de marquage comme lues, sans dépendre directement des bases de données des autres services : la création d'une notification est déclenchée applicativement par `post-svc` ou `user-svc` au moment de l'action concernée.

4.5. `feed-svc` : Fil d'actualité (Louis DROUHIN)

`feed-svc` couvre Fx5 (fil chronologique des comptes suivis). C'est le seul service de l'architecture sans base de données propre : le fil est calculé à la volée (*fan-out on read*). À chaque consultation :

1. `feed-svc` interroge `user-svc` pour obtenir la liste des comptes suivis par l'utilisateur courant ;
2. il interroge ensuite `post-svc` pour récupérer les posts de ces comptes ;
3. les résultats sont triés par date et renvoyés au client.

Ce choix de conception est documenté comme un compromis assumé plutôt qu'une contrainte technique : le *fan-out on read* est plus simple à mettre en œuvre qu'un fil matérialisé, au prix d'un coût de lecture plus élevé. Il est cohérent avec le volume de données attendu dans le cadre du projet, et identifié comme axe d'amélioration en cas de montée en charge (cf. section 6).

De plus, le fait qu'il soit dans un service dédié permet d'envisager un développement futur d'un algorithme de recommandation.

4.6. Gateway Nginx (Sébastien RUET)

La configuration de la gateway Nginx (`gateway/nginx.conf`) définit le routage vers chaque service métier ainsi que la directive `auth_request` déléguant la validation JWT à `auth-svc`, conformément au choix d'architecture présenté en section 3.5.

4.7. Front-end (Louis DROUHIN, Sébastien RUET)

Le front-end est développé en React/Next.js, avec une approche mobile-first. Il consomme exclusivement les API exposées via la gateway Nginx (jamais directement les services métier) et porte les écrans suivants : authentification (connexion, inscription), fil d'actualité, profil utilisateur, et les interactions sur les posts (publication, like, commentaire, réponse).

5. Difficultés rencontrées et solutions

5.1. Choix techniques discutés en équipe

- **Fan-out on read vs fan-out on write** pour le fil d'actualité : le choix du calcul à la volée a été préféré à un fil matérialisé pour limiter la complexité du modèle de données dans le temps imparti, au prix d'une charge de lecture plus élevée à chaque consultation du fil.
- **Modélisation des commentaires** : la tentation initiale de créer une entité `Comment` séparée de `Post` aurait limité la profondeur des fils de discussion à un seul niveau ; le choix d'unifier les deux entités sous un `Post` avec `parentId` a permis de lever cette limitation sans complexifier le modèle de données.
- **Délégation de la sécurité à la gateway** : centraliser la validation JWT dans Nginx plutôt que de la dupliquer dans chaque service métier a nécessité de figer tôt le contrat des en-têtes transmis (`X-User-Username`, `X-User-Role`) pour que tous les services s'alignent sans ambiguïté.

5.2. Points de friction inter-services

Le développement en parallèle de cinq services par des membres différents a nécessité de fixer rapidement des conventions communes (arborescence de dossiers, format des routes, gestion des erreurs) afin d'éviter une divergence de pratiques entre services qui aurait complexifié l'intégration finale via la gateway. Le `username` comme clé de cohérence partagée entre services a constitué le principal point d'attention, en l'absence de clé étrangère technique possible entre des bases de données distinctes.

6. Conclusion

6.1. Bilan par rapport aux objectifs

Le projet Breezy a permis de mettre en application l'ensemble des compétences travaillées au long de la séquence : conception argumentée d'une architecture microservices, développement d'API REST de niveau 2 de maturité, sécurisation par JWT déléguée à une gateway, développement d'un front-end React mobile-first, et conteneurisation complète de l'environnement. L'ensemble des fonctionnalités primaires du cahier des charges (Fx1 à Fx11) a été couvert, dans le périmètre explicitement retenu par l'équipe.

6.2. Axes d'amélioration et évolutions potentielles

- **Fan-out on write** pour le fil d'actualité, si la charge de lecture devenait significative, en matérialisant le fil de chaque utilisateur à l'écriture plutôt qu'à la consultation ;
- **Messagerie privée** (Fx17) et **signalement de contenu** (Fx20), retirés du périmètre initial, pour renforcer la dimension communautaire et la modération ;
- **Vidéos** sur les posts (Fx19) ;
- **Révocation explicite des jetons** (liste de révocation) plutôt que la simple suppression de la ligne en base, pour une gestion plus fine des sessions multiples ;
- **Migration vers un orchestrateur** (Kubernetes ou équivalent managé), dans la continuité de la portabilité déjà recherchée par la conteneurisation Docker, pour anticiper une montée en charge ;
- **Tests automatisés** (unitaires et d'intégration) sur chaque service, non couverts dans le temps imparti à cette itération.